

Pick the option that fits your intent. Use **Custom** if none match. Add free-form notes in the "chat more" textarea — they show up in the orange-bordered box. Click **Save answers** to write back to 05-QUESTIONS.json (uses the File System Access API in Chrome/Edge; falls back to download elsewhere). Then return to Claude and say **refresh** to process answers or **finalize** to generate 05-CONTEXT.md .

▼ Dashboard Query Design (Policies.listAssignedAndPublishedForUser)

3 / 3

Q-01 Re-acknowledgment flag derivation strategy

SPEC R3 requires `/my-policies` to render a 'Requires re-acknowledgment' badge when the user has not acked the current published version. Computation = LEFT JOIN acknowledgments ON (user_id, policy_id, policy_version_id matching current). Two implementation paths: do it in one SQL query, or split into multiple round-trips and join in app. Codebase precedent: Policies.listWithFilters is single-query with a hard LIMIT 100 (Phase 3 D-05).

- ☒ **Single SQL query with LEFT JOIN** — derive `requiresReacknowledgment` as a SELECT expression (e.g., `CASE WHEN ack.id IS NULL AND prior.id IS NOT NULL THEN true ELSE false END`)

One round-trip; whole result set is the data the page needs. Tightest perf. Slightly denser query but mirrors Policies.listWithFilters complexity. Recommended.
- ☐ **Two queries** — list assigned+published, then list this user's acks; join in app layer

Cleaner SQL on each side; can run via Promise.all to parallelize. App layer carries the join logic. Two RTTs instead of one.
- ☐ **Three queries** — assignments, then policies, then acks; join in app layer

Most decomposable; each repository method does one thing. Three RTTs. Easier to unit-test each piece in isolation but slowest user-facing.
- ☐ **Custom**

Add context, nuance, or clarification for this question...

Q-02 Department-less users (users.departmentId IS NULL) handling

Schema: `users.departmentId` is nullable; a user can have NO department membership (e.g., new hire pre-assignment, contractor). SPEC R1 acceptance test seeds 4 policies including dept-assigned ones — the seeded user has a department. Real-world employee accounts may not. The query needs an answer for the null-dept case.

- ☒ **User-only assignments visible** — WHERE clause: `(assignee\_type='user' AND assignee\_id=userId) OR (assignee\_type='department' AND assignee\_id = (SELECT department\_id FROM users WHERE id=userId))`. NULL subquery silently returns no matches.

Standard SQL behavior — `assignee\_id = NULL` is never true, so department-less users see only user-level assignments. No special-casing needed. Recommended.

- ☐ **Empty-state with helpful message** — render `/my-policies` empty + 'Your administrator hasn't assigned you to a department yet'

Trades SQL elegance for explicit UX. Forces admin onboarding flow. Risks confusing users who legitimately have no policies yet.

- ☐ **Hard error / block page render** — throw at Server Component level if departmentId is null

Strict — forces every employee to have a department. Adds complexity to onboarding. Conflicts with the existing race window (users.departmentId is settable post-webhook).

- ☐ **Custom**

Add context, nuance, or clarification for this question...

Q-03 Where the user's departmentId comes from at request time

OrgContext currently exposes {orgId, userId, clerkOrgId, clerkUserId, role} — NO departmentId. The dashboard query needs departmentId (or a SQL-level way to derive it) to filter department-assigned policies. Three places it could come from.

- ☒ **Sub-select inline in the query** — `... assignee\_id IN (SELECT department\_id FROM users WHERE id = \$userId AND org\_id = \$orgId)`

Zero extra round-trips; OrgContext stays minimal. Database does one extra index lookup per query. Schema-encoded — RLS still fires on the inner SELECT. Recommended.

- ☐ **Extend OrgContext to include departmentId** — getOrgContext does an additional users-row read, exposes departmentId alongside userId

Cleanest API surface; every Server Component gets departmentId for free. Adds 1 RTT to every getOrgContext call (already does 2 sequential DB reads for ADR-027). Useful beyond Phase 5 only if other surfaces need it.

- ☐ **Fetch fresh inside the Server Component** — call Users.findByClerkUserId(s, clerkUserId) before listAssignedAndPublishedForUser

Local to /my-policies page; doesn't change OrgContext shape. Adds 1 RTT per page request. Brittle if future pages need the same data — duplication.

- ☐ **Custom**

Add context, nuance, or clarification for this question...

▼ Acknowledgment Server Action (acknowledgePolicyAction)

5 / 5

Q-04 IP address header source

Schema: ``acknowledgments.ip_address text`` — nullable. Next.js 15 pattern: ``import { headers } from 'next/headers'; const h = await headers(); h.get('x-forwarded-for')``. Vercel populates ``x-forwarded-for`` with the user's IP first, then proxy hops. No existing IP-capture code in this repo. SPEC R2: 'IP from x-forwarded-for first hop'.

- ☒ **Strict x-forwarded-for first hop** — ``headers().get('x-forwarded-for')?.split(',')[0]?.trim() ?? null``
Matches SPEC R2 verbatim. Single header; predictable. Vercel sets this consistently. Recommended.
- ☐ **Fallback chain** — try ``cf-connecting-ip`` → ``x-real-ip`` → ``x-forwarded-for`` first hop → null
Defensive against future hosting changes (Cloudflare, custom proxy). More code; more headers to trust. Slight risk of header-spoofing via attacker-controlled cf-connecting-ip on Vercel (Vercel doesn't strip it).
- ☐ **x-real-ip only** — single non-list header
Cleaner parsing (no split). Vercel does set it. Less standard across hosts; if we ever migrate, this breaks first.
- ☐ **Custom**

Add context, nuance, or clarification for this question...

Q-05 Double-click / replay idempotency

User clicks Acknowledge twice rapidly (or browser retries). Without a guard, both clicks insert ack rows. Schema today: no UNIQUE on acknowledgments; INSERT always succeeds. ADR-018 says append-only — but duplicate acks for the same (user, policy, version) clutter the audit trail. Coupled with Q-23.

- ☒ **Schema UNIQUE on (user_id, policy_id, policy_version_id) + `INSERT ... ON CONFLICT DO NOTHING`**
DB-enforced idempotency. CLAUDE.md ASK FIRST — additive (no DROP), pre-paying-customer status per STATE.md. Audit-trail invariant: one ack per (user, version). Recommended; couples with Q-23(a).
- ☐ **Accept duplicates** — every click writes a row
No schema change. Audit reports dedupe at read time. Semantically defensible ('user clicked Acknowledge at T1 and T2'). Phase 5 surface area shrinks.
- ☐ **App-layer SELECT-then-INSERT inside the same transaction**
No schema change but race-prone unless SERIALIZABLE isolation. Two clicks landing in parallel with OrgScope transactions both see no existing row → both insert. Worst-of-both options.
- ☐ **Custom**

Add context, nuance, or clarification for this question...

Q-06 Acknowledged-while-archived handling

User loads `/my-policies/{id}` (`status=published`), admin archives between page load and click. SPEC says *Draft+Under Review* never appear — but the page didn't refresh. The ack click hits the action with a now-archived policy.

- ☒ **Reject with 400 + UI message — typed error 'This policy was archived. Refresh to update your list.'**
Honest UX; tells user the world changed. Forces a refresh; the policy disappears from `/my-policies` after refresh (archived $\neq$ assigned+published). Recommended.
- ☐ **Allow — record the ack against the last-published version; archived state doesn't invalidate the click**
Liberal write policy. Audit interpretation: 'user engaged with the policy before archival'. Could surprise audit consumers who expect acks only on published policies.
- ☐ **Silent no-op — return success without inserting; UI re-renders to the (now-empty) list**
Lies to the user (says success when no row written). Cleanest visible UX; worst audit-trail story.
- ☐ **Custom**

Add context, nuance, or clarification for this question...

Q-07 Acknowledged-while-unassigned handling

Admin removes the `policy_assignment` row between page load and click. Page still shows the policy (cached render); ack click hits the action with no current assignment.

- ☒ **Reject with 403 + UI message — 'You are no longer assigned this policy'**
Keeps audit trail honest (no ack rows for non-assigned users). Symmetric with Q-06(a) treatment. Recommended.
- ☐ **Allow — the user did see and engage with the policy; record the ack even if assignment was removed**
Liberal write policy. Audit consumer sees 'unassigned user acked' rows — has to filter or accept. Aligns with 'click intent matters' philosophy.
- ☐ **Silent no-op — same UX as (a) but doesn't expose the assignment change**
Lies to the user. Same cons as Q-06(c).
- ☐ **Custom**

Add context, nuance, or clarification for this question...

Q-08 Server Action file location

Phase 3 precedent: admin Server Actions live at ``app/(admin)/policies/{id}/actions.ts`` co-located with the page that invokes them. Phase 4 precedent: AI endpoints live at ``app/api/ai*/route.ts`` because they're HTTP-API-driven, not Server-Action-driven.

- ☒ **``app/(employee)/my-policies/{id}/actions.ts`` — co-located with detail page**
Matches Phase 3 admin precedent exactly. The action is invoked from the detail page's Acknowledge button. Recommended.

- ☐ `app/(employee)/actions.ts` — shared at the route-group level
Less file scaffolding. Future-friendly if /my-policies list page ever needs an inline Acknowledge button. Slight indirection.

Phase 5: Employee Portal — Discussion Questions

PolicyPilot · /gsd-discuss-phase 5 --full --power · 23 questions across 8 topics · SPEC.md locked (5 requirements)

23 **23** **0** **0**
TOTAL ANSWERED REMAINING WITH NOTES

[Download JSON](#)[Save answers](#)

▼ Re-Acknowledgment UI

2 / 2

Q-09 Re-ack indicator visual treatment

SPEC R3: 'UI renders a Requires re-acknowledgment badge (visually distinct shadcn Badge variant) when requiresReacknowledgment === true AND prior acknowledgments exist'. Phase 3 ships PolicyStatusBadge (variant for each PolicyStatus). Phase 4 ships ConsistencyFindingsList with severity badges. Shadcn Badge supports custom variants via cva.

- ☒ **Shadcn Badge with new 'warning' variant — orange-amber background + 'Requires re-acknowledgment' text**
High contrast; scannable in a list. Matches PolicyStatusBadge precedent (Phase 3). Variant can be reused for Phase 7 review-due indicators. Recommended.
- ☐ **Inline AlertCircle icon + small amber text — lower visual weight**
Subtle; doesn't visually dominate the list. Could be missed if user scans quickly. Better for power users; worse for compliance-driven UX.
- ☐ **Card left-border accent (amber 4px) + small badge — highest prominence**
Loud. Hard to miss. Could feel alarmist if many policies update at once. Visually heavy for compliance-officer dashboards.
- ☐ **Custom**

Add context, nuance, or clarification for this question...

Q-10 First-time ack vs re-ack differentiation

SPEC R3: 'badge differentiates re-ack from first-time-ack; first-time acks get a plain Acknowledge CTA, not the badge'. Three patterns of how strong that differentiation should be.

- ☒ **Badge ONLY on re-ack — first-time = clean card with 'Acknowledge' button; re-ack = warning badge + 'Re-acknowledge' button**

Per-SPEC R3 minimum; cleanest. New users see uncluttered state. Recommended.

- ☐ **Same UI for both — every unacknowledged version shows the same 'Acknowledge' CTA**

Simpler. Conflicts with SPEC R3 wording — would require SPEC amendment.

- ☐ **Both badged, different colors — first-time = blue 'Action needed'; re-ack = amber 'Re-acknowledge required'**

Symmetric; signals urgency on re-ack. More visual noise in a long list of new-employee assignments.

- ☐ **Custom**

Add context, nuance, or clarification for this question...

▼ Admin Bulk-Assignment UI

4 / 4

Q-11 Assignment surface location

SPEC: thin admin assignment surface on /policies/[id]. Phase 3 ships /policies/[id]/page.tsx with PolicyView + PolicyTransitionMenu. Three places to add the assign affordance.

- ☒ **Inline panel at bottom of /policies/[id] — dept selector + 'Assign' button below the transition menu**

Single-page workflow. Admin sees policy + transitions + assignments together. Recommended.

- ☐ **Modal dialog triggered by 'Assign' button — shadcn Dialog with dept selector**

Keeps detail page focused on policy content. Slightly more clicks per assignment. Better for low-frequency action; worse for setting up a new policy with multiple assignments.

- ☐ **Dedicated /policies/[id]/assignments page — full route**

Future-proof for managing many assignments. Heavier; out-of-scope-feeling for Phase 5's thin scope. Decoupling forces a second navigation step.

- ☐ **Custom**

Add context, nuance, or clarification for this question...

Q-12 Empty-departments UX

Departments are created via Departments.create — currently a throw-stub (Phase 3+ admin user management). A fresh org has zero departments. Phase 5 admin tries to bulk-assign — no targets.

- ☒ **Disable assignment button + tooltip — 'Create a department first'; link to admin settings (when it exists)**
Doesn't pretend departments are available. No risk of phantom selector. Defers department-create UX to a future phase. Recommended.
- ☐ **Inline 'Create department' flow — small + button next to the selector lets admin create dept inline**
Lets admin do the whole flow from one page. Requires shipping `Departments.create()` body AND a UI affordance in Phase 5 — scope creep.
- ☐ **Hide assignment surface entirely until at least one department exists**
Cleanest UI. Conceals the affordance — admin doesn't learn it exists until departments do. Worse discoverability.
- ☐ **Custom**

Add context, nuance, or clarification for this question...

Q-13 Duplicate-assignment guard (application layer)

*Admin clicks 'Assign to Dept D' twice. Schema today has no **UNIQUE** on `policy_assignments`. App layer decides what happens. Coupled with Q-22 (schema decision).*

- ☒ **DB **UNIQUE** on (`policy_id`, `assignee_type`, `assignee_id`) + 'ON CONFLICT DO NOTHING'**
Schema-enforced; second click is a silent no-op via UPSERT. CLAUDE.md ASK FIRST trigger (couples with Q-22(a)). SPEC R4 acceptance test 'count = 1, not N' passes naturally. Recommended.
- ☐ **App-layer check before insert — call `PolicyAssignments.listForPolicy` first; skip insert if duplicate found**
No schema change. Race-prone — two parallel admin clicks both see no row, both insert. Imperfect for SPEC R4's 'exactly one row' invariant.
- ☐ **Accept duplicates — store both rows; query uses **DISTINCT** or **IN****
Simplest backend. Conflicts with SPEC R4 ('exactly one row'). Adds query complexity downstream.
- ☐ **Custom**

Add context, nuance, or clarification for this question...

Q-14 Un-assign affordance scope

SPEC In-Scope: 'thin admin bulk-dept-assign UI'. Out-of-Scope: 'individual-user assignment admin UI'. SPEC does NOT explicitly include or exclude un-assign.

- ☒ **Out of scope for Phase 5 — read-only list of assignments; un-assign deferred to a follow-up**
Tightest scope. Admin gets stuck if they assign wrong dept. Workaround: edit-policy + re-publish forces re-ack on the new version anyway. Recommended (minimum-viable).
- ☐ **In scope — same surface gets a 'Remove' button + confirm dialog; deletes `policy_assignments` row**

Symmetric with assign. Introduces the only DELETE in the assignment path. Code is small but adds a delete-confirm dialog + test coverage.

- ☐ **In scope but soft-delete only — add `revokedAt` column to policy_assignments + filter on it**
Most audit-friendly (preserves history). Bigger schema change. Slippery slope toward 'why don't acks have revokedAt too'.
- ☐ **Custom**

Add context, nuance, or clarification for this question...

▼ Append-Only CI Gate (scripts/check-acknowledgment-immutability.ts)

3 / 3

Q-15 Gate implementation approach

SPEC R5 requires a new ts-morph-grade CI gate forbidding ``update(acknowledgments)`` / ``delete(acknowledgments)`` outside the type-test fixture. Codebase precedents: `scripts/check-policy-id-brand.ts` + `check-error-discipline.ts` use `ts-morph` for type-aware AST traversal; `scripts/check-artifacts.ts` uses `regex` with `comment-stripping` (PR #7 ADR-027 pattern).

- ☒ **ts-morph AST traversal — Project.addSourceFilesAtPaths, walk CallExpression nodes, check method-chain against the `acknowledgments` schema import**
Type-aware. Resilient to formatting / comment shifts. Catches ``s.tx.update(acknowledgments)`` even with whitespace variations. Recommended.
- ☐ **Regex with comment-stripping — strip ``//`` and ``/* */`` then test ``\update\`` `(acknowledgments)\`` `\.delete\`` `(acknowledgments)\``**
Lighter dep footprint (no `ts-morph` in this script — but `ts-morph` already installed). Misses aliased imports (e.g., ``import { acknowledgments as ack }``). Faster to read.
- ☐ **Hybrid — ts-morph for repository file + lib/policies/transitions.ts; regex sweep for everything else**
Most thorough. Two code paths to maintain. Premature optimization given Phase 5's small surface.
- ☐ **Custom**

Add context, nuance, or clarification for this question...

Q-16 Scope of the scan

The repository (`lib/db/repositories/acknowledgments.ts`) is the only legitimate site for any write to ``acknowledgments``. Other files importing the schema would be caught by Phase 2's `check-db-imports.ts` (raw-db allow-list). Question: how wide a net does this gate cast?

- ☒ **Whole ``lib/**/*.ts`` excluding `tests/types.ts` fixture + `tests/fixtures/*` — broad sweep**

Defense-in-depth: catches any future code that smuggles in a write via a helper module or orchestrator. Recommended.

- ☐ **Repository file only** (`lib/db/repositories/acknowledgments.ts`)

Tightest. Relies on check-db-imports.ts to keep `acknowledgments` schema-import out of other files. Two gates layered.
- ☐ **Whole repo** `\*\*/\*.ts` excluding tests/ + scripts/ — **most paranoid**

Catches even app/(admin)/policies/[id]/actions.ts (no legit reason to write acks there). Most coverage; some false-positive risk on intentional dev fixtures.
- ☐ **Custom**

Add context, nuance, or clarification for this question...

Q-17 Negative-control fixture wiring

SPEC R5 acceptance requires the gate to exit non-zero against a deliberately-violating fixture — proves the gate is non-vacuous. Phase 4 precedent: check-ai-prompts has scratch/probe.ts to verify prompts work; check-ai-layer runs as vitest. Phase 2 precedent: check-rls seeds a TEST DB and rolls back.

- ☒ **Sibling fixture under `tests/fixtures/ack-mutation-attempt.ts` + spawn the gate as subprocess; assert non-zero exit**

End-to-end smoke. Verifies the actual binary catches the violation. Mirrors Phase 4 SDK→SPEC translator test pattern. Recommended.
- ☐ **Vitest unit test with inline-string fixture — gate exposes a testable function; test passes a TS source string and asserts the verdict**

Faster (no subprocess). Requires the gate's logic to be exportable. Tests the logic; doesn't test the CLI wiring.
- ☐ **No negative-control test — trust the gate code**

Lightest weight. Risk: gate could break silently (e.g., a refactor that drops the actual scan call). Phase 5 SPEC R5 acceptance explicitly demands proof of non-vacuous — option (c) doesn't satisfy SPEC.
- ☐ **Custom**

Add context, nuance, or clarification for this question...

▼ Test Strategy & verify:phase-5 Chain

3 / 3

Q-18 Vitest organization for Phase 5 tests

Phase 3 precedent: lib/policies/state-machine.test.ts and lib/auth/bootstrap-errors.test.ts are co-located with source. Phase 4 precedent: scripts/check-ai-layer.test.ts is centralized at scripts/. SPEC R1/R3/R4 require integration tests

against real DB.

- ☒ **Co-located unit tests + scripts/ integration tests — ``lib/db/repositories/acknowledgments.test.ts`` (unit, mocked) + ``scripts/check-employee-portal.ts`` (integration, real TEST DB)**
Matches both Phase 3 (co-located units) and Phase 4 (scripts integration) precedents. Recommended.
- ☐ **Centralized under ``tests/phase-5/`` — all Phase-5-specific tests live in one dir; integration vs unit by file naming**
Easy to scope verify:phase-5. Diverges from Phase 3 layout pattern. New convention to maintain.
- ☐ **All co-located — even integration tests live next to the file under test; vitest config separates by extension**
Strictest co-location. Integration tests pollute source dirs (real-DB tests can be slow; co-locating signals they're not pure unit).
- ☐ **Custom**

Add context, nuance, or clarification for this question...

Q-19 Integration test DB strategy

SPEC R1 acceptance test seeds 4 policies + asserts query returns exactly P1 + P2. SPEC R3 acceptance test acks v1, republishes, asserts re-ack badge. SPEC R4 asserts 3 dept members see policy. Phase 2/3 precedents: `scripts/check-rls.ts` and `scripts/check-policies-list-filters.ts` seed via `postgres-js` raw + ROLLBACK at end.

- ☒ **Raw postgres-js + BYPASSRLS seed + SET LOCAL ROLE authenticated for the test + ROLLBACK at end — matches check-rls + check-policies-list-filters precedent**
Exercises RLS + schema constraints exactly as production does. Cross-org isolation testable. Slower (~1-2s per script). Recommended.
- ☐ **Vitest + Drizzle on TEST DB — beforeAll seeds via Drizzle, afterAll cleans; same TEST DB but cleaner API**
Less raw SQL. Doesn't naturally test RLS unless you also configure jwt claims. Mixes ORM and test concerns.
- ☐ **Mocks for everything — vitest with `vi.mock('@lib/db')` returning canned rows**
Fast. Doesn't exercise RLS, FK integrity, or schema constraints. SPEC R5 cross-org isolation test (acceptance) NOT verifiable. Insufficient for SPEC.
- ☐ **Custom**

Add context, nuance, or clarification for this question...

Q-20 verify:phase-5 chain composition

Current: ``verify:phase-4 = verify:phase-3 && check:ai-prompts && check:ai-layer``. Phase 5 needs at least: `typecheck`, all Phase 3/4 gates, `check-acknowledgment-immutability`, `check-employee-portal` (integration test).

- ☒ **`pnpm verify:phase-4 && pnpm check:acknowledgment-immutability && pnpm check:employee-portal` — chain forward**
Cumulative: every Phase-1-through-5 gate runs. Slowest but most defensive. Phase 4 already chains Phase 3. Recommended.
- ☐ **Discrete chain — `verify:phase-5 = typecheck + check:artifacts + check:ack-immutability + check:employee-portal + test`; skip Phase 3/4 gates**
Fast fail signal for Phase-5-specific drift. Misses regressions in Phase 3/4 paths. Worse for ship-readiness.
- ☐ **Hybrid — `verify:phase-5 = verify:phase-4 && [Phase 5 gates]`; add separate `verify:phase-5-fast` for inner-loop development**
Best of both. Two scripts to maintain. Inner-loop variant deviates from one-script-to-verify-a-phase convention.
- ☐ **Custom**

Add context, nuance, or clarification for this question...

▼ Phase 4 Q&A Surface Reconciliation

1 / 1

Q-21 Employee-side Q&A affordance

ROADMAP Phase 5 goal text: 'employee can ... ask Q&A questions'. ADR-029 (2026-05-21): 'Phase 5 SC 1-5 do not consume Phase 4 AI surfaces'. SPEC R-boundaries puts employee Q&A UI explicitly OUT OF SCOPE. POST /api/ai/qa exists from Phase 4 and is reachable by any authenticated user. Decision: do we hint at the future surface in /my-policies, or omit entirely?

- ☐ **Omit entirely — no Q&A link, button, or sidebar item in /my-policies. ADR-029 + SPEC boundaries are binding; goal-narrative reference is informational only.**
Tightest. Zero scope creep. Future phase can add Q&A as a clean surface. Recommended.
- ☐ **Disabled placeholder link — sidebar/topbar shows 'Ask the AI' with 'Coming soon' tooltip + cursor-not-allowed**
Documents future affordance. Minor scope creep (UI element shipped that does nothing). Risk: users click and feel teased.
- ☒ **Wire it through — ship thin /my-policies/ask page calling existing POST /api/ai/qa**
Honors goal-narrative. Direct scope-creep vs ADR-029 + SPEC. Adds untested-in-Phase-5 attack surface (Q&A UI security review skipped).
- ☐ **Custom**

Add context, nuance, or clarification for this question...

▼ Schema Safety — UNIQUE Constraints

2 / 2

Q-22 **UNIQUE on policy_assignments(policy_id, assignee_type, assignee_id)**

Currently no UNIQUE on policy_assignments. SPEC R4 demands 'exactly one row' for dept assignment. CLAUDE.md ASK FIRST: schema changes after Phase 2 require operator approval. Pre-paying-customer per STATE.md → additive constraints are safe. Coupled with Q-13.

- ☒ **Add it — new migration `0010\_policy\_assignments\_unique.sql` (additive). Mirror 03-G3 T2/T3 belt-and-suspenders pattern for policy_versions**
Schema-enforced 'exactly one row' invariant. Phase 5 SC #4 trivially satisfied. ASK FIRST satisfied: additive + pre-customer. Recommended; couples with Q-13(a).
- ☐ **Skip — rely on app-layer dedup (Q-13(b)) or accept duplicates (Q-13(c))**
No schema change. Phase 5 surface area smaller. Risk: SPEC R4 'exactly one row' invariant is app-enforced only — easier to regress in future phases.
- ☐ **Add but defer to Phase 5.5 polish — track in a follow-up issue like Phase 4 issue #16**
Phase 5 ships without; backlog catches it later. Pattern works if there's a clear backlog process. Slight risk of indefinite deferral.
- ☐ **Custom**

Add context, nuance, or clarification for this question...

Q-23 **UNIQUE on acknowledgments(user_id, policy_id, policy_version_id)**

Currently no UNIQUE on acknowledgments. SPEC R5 + ADR-018 says append-only; doesn't say 'one ack per version'. But duplicate acks for the same (user, version) bloat the audit trail. Coupled with Q-05.

- ☒ **Add it — new migration; enables `ON CONFLICT DO NOTHING` on insert. Enforces 'one ack per (user, version)' at DB level**
DB-enforced idempotency. ASK FIRST: additive + pre-customer. Couples with Q-05(a). Audit-trail invariant strengthened. Recommended.
- ☐ **Skip — accept duplicates; audit consumers dedupe at read time**
No schema change. Couples with Q-05(b). 'Audit trail captures every click' interpretation.
- ☐ **Add but defer — Phase 5.5 polish**
Same defer pattern as Q-22(c).
- ☐ **Custom**

Add context, nuance, or clarification for this question...

Saved at 12:26:09

Download JSON

Save answers